

Süreçler, Thread'ler ve Context Switch

İşletim Sistemleri — Konu 1

1. Giriş: Süreç Nedir?

Bir **süreç (process)**, çalışmakta olan bir programın işletim sistemi tarafından yönetilen soyutlamasıdır. Diskteki bir program dosyası pasiftir; işletim sistemi onu belleğe yükleyip bir süreç olarak başlattığında aktif hale gelir. Her sürecin kendine ait bir adres uzayı, açık dosya tanıtıcıları (file descriptor) listesi ve bir veya daha fazla **thread**'i vardır.

Bir sürecin işletim sistemi tarafından tutulan durumu **Process Control Block (PCB)** içinde saklanır. PCB şunları içerir:

- Süreç kimliği (PID)
- Süreç durumu (running, ready, blocked, terminated)
- Program sayacı (program counter) ve CPU yazmaçlarının (register) o anki değerleri
- Bellek yönetim bilgisi (sayfa tablosu işaretçisi)
- Açık dosya tablosu
- Zamanlama (scheduling) bilgisi: öncelik, kullanılan CPU süresi

1.1. Süreç Durumları

Bir süreç yaşam döngüsü boyunca birkaç durum arasında geçiş yapar:

```
new -> ready -> running -> terminated
      ^           |
      |           v
      +----- blocked
```

- **ready**: Süreç çalışmaya hazır, CPU'yu bekliyor.
- **running**: Süreç şu an CPU üzerinde çalışıyor.
- **blocked (waiting)**: Süreç bir G/Ç (I/O) işleminin veya bir olayın tamamlanmasını bekliyor; CPU'yu kullanamaz.

2. Thread Nedir, Süreçten Farkı Ne?

Bir **thread**, bir süreç içinde bağımsız olarak zamanlanabilen en küçük yürütme birimidir. Aynı sürece ait thread'ler adres uzayını, açık dosyaları ve global değişkenleri

paylaşır; ama her thread'in kendine ait bir yığın (stack), program sayacı ve yazmaç kümesi vardır.

Özellik	Süreç	Thread
Adres uzayı	Kendine özel	Süreç içindeki diğer thread'lerle paylaşılır
Oluşturma maliyeti	Yüksek (yeni adres uzayı, sayfa tablosu)	Düşük
İletişim	IPC gerekir (pipe, soket, paylaşımli bellek)	Doğrudan paylaşılan bellek
Bir tanesi çökerse	Diğer süreçleri etkilemez	Aynı sürecin diğer thread'lerini etkileyebilir

Çok thread'li bir programda thread'ler arası paylaşılan veriye erişim senkronizasyon gerektirir (bkz. Konu 4: Senkronizasyon — mutex, semafor).

3. Süreç Yaratma: fork()

Unix/Linux sistemlerinde yeni bir süreç yaratmanın klasik yolu fork() sistem çağrısıdır. fork() çağrıldığında işletim sistemi, çağıran sürecin (ebeveyn/parent) neredeyse birebir kopyası olan yeni bir süreç (çocuk/child) yaratır.

```
pid_t pid = fork();
if (pid == 0) {
    // çocuk süreçte çalışır
} else if (pid > 0) {
    // ebeveyn süreçte çalışır; pid çocuğun PID'sidir
} else {
    // fork() başarısız oldu
}
```

Önemli noktalar:

- fork() **iki kez döner**: ebeveynde çocuğun PID'siyle, çocukta 0 ile.
- Çocuk, ebeveynin adres uzayının bir kopyasını alır (modern sistemlerde **copy-on-write** ile — fiziksel kopyalama yalnız bir sayfa yazıldığında olur, bu da fork()'u ucuzlatır).
- Çocuk, ebeveynin açık dosya tanımlayıcılarını da **miras alır**. Bir dosya tanımlayıcısı fork() sonrası kapatılmazsa, iki süreç aynı dosyaya erişmeye devam eder ve bu bazen kaynak sızıntısına (dosya tanıtıcı sızıntısı) yol açar — bkz. sample_data/isletim-sistemleri/fork_example.c dosyasındaki örnek.
- Ebeveyn genellikle wait() çağırarak çocuğun bitmesini bekler; aksi halde çocuk bittiğinde **zombi süreç** olarak kalabilir.

4. Context Switch

Bir CPU çekirdeği aynı anda yalnız bir thread çalıştırabilir. Zamanlayıcı (scheduler) çalışan thread'i değiştirmeye karar verdiğinde bir **context switch** gerçekleşir:

1. Çalışmakta olan thread'in yazmaçları, program sayacı ve yığın işaretçisi onun PCB'sine kaydedilir.
2. Zamanlayıcı bir sonraki çalıştırılacak thread'i seçer (bkz. Konu 2: CPU Zamanlama).
3. Seçilen thread'in kayıtlı durumu yazmaçlara geri yüklenir.
4. Süreçler arası context switch'te ayrıca **bellek yönetim birimi (MMU)** yeni sürecin sayfa tablosuna işaret edecek şekilde güncellenir ve genellikle **TLB** (Translation Lookaside Buffer, bkz. Konu 3) geçersiz kılınır (flush) — bu da context switch'i aynı süreç içindeki iki thread arasında geçişten daha pahalı yapar.

Context switch **saf ek yüküdür (overhead)**: bu sırada hiçbir kullanıcı kodu çalışmaz. Zamanlayıcının kullandığı zaman dilimi (quantum) çok küçük seçilirse context switch sıklığı artar ve toplam verimlilik düşer — bu ilişki Konu 2'de round-robin zamanlamasının quantum seçimi tartışılırken ayrıntılandırılır.

5. Özet

- Süreç = adres uzayı + kaynaklar; thread = süreç içinde zamanlanabilir yürütme birimi.
- Thread'ler bellek paylaşır, süreçler paylaşmaz; bu paylaşım hem ucuzluk hem senkronizasyon riski getirir.
- `fork()` bir süreci kopyalar; açık dosya tanımlayıcıları miras alınır ve kapatılmazsa sızıntıya yol açabilir.
- Context switch, çalışan thread'i değiştirmenin donanım+yazılım maliyetidir; süreçler arası context switch, TLB flush nedeniyle thread'ler arasındakinden daha pahalıdır.